

Proyecto 3 - Othello

Marcos Díaz (221102), Daniel Machic (22118), Maria Jose Ramírez (221051)

GitHub: <https://github.com/MarcosDiaz1409/Proyecto3-IA.git>

Importar Librerías

```
In [ ]: import numpy as np
import pandas as pd
import pygame
import time
import math
import random
from copy import deepcopy
```

Representación del tablero

```
In [ ]: EMPTY=0
BLACK=1
WHITE=-1
```

Clase OthelloEngine

Esta clase `OthelloEngine` constituye el núcleo lógico del juego. Es responsable de gestionar el estado del tablero, validar movimientos, aplicar las reglas de Othello, realizar el volteo de fichas y determinar cuándo una partida ha finalizado. Esta separación permite reutilizar la lógica del juego independientemente de la interfaz gráfica o del agente inteligente utilizado.

```
In [ ]: class OthelloEngine:

    EMPTY = 0
    BLACK = 1
    WHITE = -1

    DIRECTIONS = [
        (-1, -1), (-1, 0), (-1, 1),
        (0, -1),      (0, 1),
        (1, -1),  (1, 0),  (1, 1)
    ]

    def __init__(self):
```

```

self.board = np.zeros((8, 8), dtype=int)

# Posición inicial oficial
self.board[3][3] = self.WHITE
self.board[3][4] = self.BLACK
self.board[4][3] = self.BLACK
self.board[4][4] = self.WHITE

self.current_player = self.BLACK

# =====
# UTILIDADES
def is_on_board(self, row, col):
    return 0 <= row < 8 and 0 <= col < 8

def get_opponent(self, player):
    return -player

def clone(self):

    new_game = OthelloEngine()

    new_game.board = self.board.copy()
    new_game.current_player = self.current_player

    return new_game

# =====
# MOVIMIENTOS
def get_flips(self, row, col, player):

    if not self.is_on_board(row, col):
        return []

    if self.board[row][col] != self.EMPTY:
        return []

    opponent = self.get_opponent(player)

    flips = []

    for dr, dc in self.DIRECTIONS:

        r = row + dr
        c = col + dc

        temp_flips = []

        while (
            self.is_on_board(r, c)
            and self.board[r][c] == opponent
        ):

            temp_flips.append((r, c))

            r += dr

```

```

        c += dc

        if (
            len(temp_flips) > 0
            and self.is_on_board(r, c)
            and self.board[r][c] == player
        ):
            flips.extend(temp_flips)

    return flips

def is_valid_move(self, row, col, player):

    return len(self.get_flips(row, col, player)) > 0

def get_legal_moves(self, player):

    legal_moves = []

    for row in range(8):
        for col in range(8):

            if self.is_valid_move(row, col, player):
                legal_moves.append((row, col))

    return legal_moves

def has_valid_move(self, player):

    return len(self.get_legal_moves(player)) > 0

def apply_move(self, row, col, player):

    flips = self.get_flips(row, col, player)

    if len(flips) == 0:
        return False

    self.board[row][col] = player

    for r, c in flips:
        self.board[r][c] = player

    self.current_player = self.get_opponent(player)

    return True

def pass_turn(self):

    self.current_player = self.get_opponent(
        self.current_player
    )

# =====
# ESTADO DEL JUEGO
def count_discs(self):

```

```

        black_count = np.sum(
            self.board == self.BLACK
        )

        white_count = np.sum(
            self.board == self.WHITE
        )

        return black_count, white_count

    def is_board_full(self):

        return not np.any(
            self.board == self.EMPTY
        )

    def is_terminal(self):

        return (
            self.is_board_full()
            or
            (
                not self.has_valid_move(self.BLACK)
                and
                not self.has_valid_move(self.WHITE)
            )
        )

    def get_winner(self):

        black_count, white_count = self.count_discs()

        if black_count > white_count:
            return self.BLACK

        elif white_count > black_count:
            return self.WHITE

        return 0

# =====
# DEBUG / VISUALIZACIÓN
    def print_board(self):

        symbols = {
            self.EMPTY: ".",
            self.BLACK: "B",
            self.WHITE: "W"
        }

        print("  0 1 2 3 4 5 6 7")

        for r in range(8):

            row_string = f"{r} "

```

```

        row_string += " ".join(
            symbols[cell]
            for cell in self.board[r]
        )

        print(row_string)

    def print_score(self):

        black_count, white_count = self.count_discs()

        print(
            f"Black: {black_count} | "
            f"White: {white_count}"
        )

```

Heurísticas por fases

Debido a la complejidad combinatoria de Othello, no es factible explorar el árbol completo de juego en la mayoría de situaciones. Por esta razón se implementa una función heurística que evalúa estados no terminales considerando factores como movilidad, control de esquinas, control de bordes y posiciones estratégicas. Además, los pesos de evaluación cambian según la fase de la partida para aproximar estrategias utilizadas por jugadores competitivos.

```

In [ ]: class OthelloHeuristic:

    CORNERS = [
        (0, 0),
        (0, 7),
        (7, 0),
        (7, 7)
    ]

    X_SQUARES = [
        (1, 1),
        (1, 6),
        (6, 1),
        (6, 6)
    ]

    C_SQUARES = [
        (0, 1), (1, 0),
        (0, 6), (1, 7),
        (6, 0), (7, 1),
        (6, 7), (7, 6)
    ]

    @staticmethod
    def evaluate(game, player):

```

```
opponent = game.get_opponent(player)

# Diferencia de fichas
player_discs = np.sum(game.board == player)
opponent_discs = np.sum(game.board == opponent)

disc_score = player_discs - opponent_discs

# Movilidad
player_moves = len(
    game.get_legal_moves(player)
)

opponent_moves = len(
    game.get_legal_moves(opponent)
)

mobility_score = (
    player_moves - opponent_moves
)

# Esquinas
corner_score = 0

for r, c in OthelloHeuristic.CORNERS:

    if game.board[r][c] == player:
        corner_score += 1

    elif game.board[r][c] == opponent:
        corner_score -= 1

# X Squares
x_score = 0

for r, c in OthelloHeuristic.X_SQUARES:

    if game.board[r][c] == player:
        x_score -= 1

    elif game.board[r][c] == opponent:
        x_score += 1

# C Squares
c_score = 0

for r, c in OthelloHeuristic.C_SQUARES:

    if game.board[r][c] == player:
        c_score -= 1

    elif game.board[r][c] == opponent:
        c_score += 1

# Bordes
edge_score = 0
```

```
for i in range(8):

    positions = [
        (0, i),
        (7, i),
        (i, 0),
        (i, 7)
    ]

    for r, c in positions:

        if game.board[r][c] == player:
            edge_score += 1

        elif game.board[r][c] == opponent:
            edge_score -= 1

# Fase del juego
occupied = np.count_nonzero(
    game.board
)

# APERTURA
if occupied < 20:

    score = (
        5 * mobility_score
        + 30 * corner_score
        + 3 * edge_score
        + 10 * x_score
        + 8 * c_score
        + 1 * disc_score
    )

# MEDIO JUEGO
elif occupied < 50:

    score = (
        4 * mobility_score
        + 40 * corner_score
        + 5 * edge_score
        + 8 * x_score
        + 6 * c_score
        + 2 * disc_score
    )

# FINAL
else:

    score = (
        2 * mobility_score
        + 40 * corner_score
        + 3 * edge_score
        + 4 * x_score
        + 2 * c_score
```

```

        + 10 * disc_score
    )

    return score

```

Alpha-Beta

Se implementa un agente basado en Minimax con poda Alpha-Beta. Este algoritmo explora el árbol de decisiones buscando maximizar la utilidad del jugador actual y minimizar la del oponente. La poda Alpha-Beta reduce significativamente la cantidad de estados evaluados sin afectar la calidad de la decisión final, permitiendo alcanzar mayores profundidades de búsqueda dentro de las restricciones temporales del proyecto.

```

In [ ]: class AlphaBetaAgent:

    def __init__(self, depth=4):

        self.depth = depth

        self.nodes_explored = 0

        self.last_search_time = 0

    # Movimiento principal
    def get_move(self, game):

        start_time = time.time()

        self.nodes_explored = 0

        player = game.current_player

        legal_moves = game.get_legal_moves(player)

        if not legal_moves:
            return None

        best_move = None

        best_value = -math.inf

        alpha = -math.inf
        beta = math.inf

        for move in legal_moves:

            new_game = game.clone()

            new_game.apply_move(
                move[0],
                move[1],
                player
            )

```



```

        value = self.min_value(
            new_game,
            self.depth - 1,
            alpha,
            beta,
            player
        )

        if value > best_value:

            best_value = value
            best_move = move

            alpha = max(alpha, best_value)

    self.last_search_time = (
        time.time() - start_time
    )

    return best_move

# MAX
def max_value(
    self,
    game,
    depth,
    alpha,
    beta,
    maximizing_player
):

    self.nodes_explored += 1

    if (
        depth == 0
        or game.is_terminal()
    ):

        return OthelloHeuristic.evaluate(
            game,
            maximizing_player
        )

    current_player = maximizing_player

    legal_moves = game.get_legal_moves(
        current_player
    )

    if not legal_moves:

        return self.min_value(
            game,
            depth - 1,
            alpha,

```

```

        beta,
        maximizing_player
    )

    value = -math.inf

    for move in legal_moves:

        child = game.clone()

        child.apply_move(
            move[0],
            move[1],
            current_player
        )

        value = max(
            value,
            self.min_value(
                child,
                depth - 1,
                alpha,
                beta,
                maximizing_player
            )
        )

        alpha = max(alpha, value)

        if alpha >= beta:
            break

    return value

# MIN
def min_value(
    self,
    game,
    depth,
    alpha,
    beta,
    maximizing_player
):

    self.nodes_explored += 1

    if (
        depth == 0
        or game.is_terminal()
    ):

        return OthelloHeuristic.evaluate(
            game,
            maximizing_player
        )

```

```

current_player = game.get_opponent(
    maximizing_player
)

legal_moves = game.get_legal_moves(
    current_player
)

if not legal_moves:

    return self.max_value(
        game,
        depth - 1,
        alpha,
        beta,
        maximizing_player
    )

value = math.inf

for move in legal_moves:

    child = game.clone()

    child.apply_move(
        move[0],
        move[1],
        current_player
    )

    value = min(
        value,
        self.max_value(
            child,
            depth - 1,
            alpha,
            beta,
            maximizing_player
        )
    )

    beta = min(beta, value)

    if alpha >= beta:
        break

return value

```

Agente Minimax

Como referencia para el análisis de rendimiento, se implementa una versión de Minimax sin poda. Este agente permite comparar directamente el crecimiento del árbol de búsqueda frente a Alpha-Beta y analizar el impacto de la poda sobre la cantidad de nodos explorados.

```
In [ ]: class MinimaxAgent:

    def __init__(self, depth=4):

        self.depth = depth

        self.nodes_explored = 0

        self.last_search_time = 0

    def get_move(self, game):

        start_time = time.time()

        self.nodes_explored = 0

        player = game.current_player

        best_move = None

        best_value = -math.inf

        for move in game.get_legal_moves(
            player
        ):

            child = game.clone()

            child.apply_move(
                move[0],
                move[1],
                player
            )

            value = self.min_value(
                child,
                self.depth - 1,
                player
            )

            if value > best_value:

                best_value = value

                best_move = move

        self.last_search_time = (
            time.time()
            - start_time
        )

        return best_move

    def max_value(
        self,
```

```

        game,
        depth,
        maximizing_player
    ):

        self.nodes_explored += 1

        if (
            depth == 0
            or game.is_terminal()
        ):

            return OthelloHeuristic.evaluate(
                game,
                maximizing_player
            )

        value = -math.inf

        for move in game.get_legal_moves(
            maximizing_player
        ):

            child = game.clone()

            child.apply_move(
                move[0],
                move[1],
                maximizing_player
            )

            value = max(
                value,
                self.min_value(
                    child,
                    depth - 1,
                    maximizing_player
                )
            )

        return value

    def min_value(
        self,
        game,
        depth,
        maximizing_player
    ):

        self.nodes_explored += 1

        if (
            depth == 0
            or game.is_terminal()
        ):

```

```

        return OthelloHeuristic.evaluate(
            game,
            maximizing_player
        )

    opponent = game.get_opponent(
        maximizing_player
    )

    value = math.inf

    for move in game.get_legal_moves(
        opponent
    ):

        child = game.clone()

        child.apply_move(
            move[0],
            move[1],
            opponent
        )

        value = min(
            value,
            self.max_value(
                child,
                depth - 1,
                maximizing_player
            )
        )

    return value

```

Monte Carlo Tree Search (MCTS)

Se implementa un agente basado en Monte Carlo Tree Search (MCTS), una técnica de búsqueda que estima la calidad de los movimientos mediante simulaciones repetidas de partidas. El algoritmo utiliza la estrategia UCT (Upper Confidence Bound applied to Trees) para equilibrar exploración y explotación durante la construcción del árbol de búsqueda. Este agente será comparado experimentalmente contra Alpha-Beta en el torneo IA vs IA requerido por el proyecto.

Nodo MCTS

```

In [ ]: class MCTSNode:

    def __init__(
        self,
        game,
        parent=None,

```

```
        move=None
    ):

        self.game = game

        self.parent = parent

        self.move = move

        self.children = []

        self.visits = 0

        self.wins = 0

        self.untried_moves = (
            game.get_legal_moves(
                game.current_player
            )
        )

    def is_fully_expanded(self):

        return (
            len(self.untried_moves) == 0
        )

    def best_child(self, c=1.414):

        best_score = -float("inf")

        best_node = None

        for child in self.children:

            exploitation = (
                child.wins /
                child.visits
            )

            exploration = c * math.sqrt(
                math.log(self.visits)
                / child.visits
            )

            score = (
                exploitation
                + exploration
            )

            if score > best_score:

                best_score = score

                best_node = child
```

```
return best_node
```

Agente MCTS

```
In [ ]: class MCTSAgent:

    def __init__(
        self,
        time_limit=2.0,
        exploration_constant=1.414
    ):

        self.time_limit = time_limit
        self.C = exploration_constant
        self.last_search_time = 0
        self.iterations_done = 0

    def get_move(self, game):

        start_time = time.time()
        self.iterations_done = 0

        root = MCTSNode(
            game.clone()
        )

        root_player = (
            game.current_player
        )

        while (
            time.time() - start_time
            < self.time_limit
        ):

            self.iterations_done += 1
            node = root

            state = (
                game.clone()
            )

            # Selection
            while (
                node.is_fully_expanded()
                and node.children
            ):

                node = node.best_child(
                    self.C
                )

            state.apply_move(
                node.move[0],
```



```

        node.move[1],
        state.current_player
    )

    # Expansion
    if node.untried_moves:

        move = random.choice(
            node.untried_moves
        )

        node.untried_moves.remove(
            move
        )

        state.apply_move(
            move[0],
            move[1],
            state.current_player
        )

        child = MCTSNode(
            state.clone(),
            parent=node,
            move=move
        )

        node.children.append(
            child
        )

        node = child

    # Simulation
    rollout_state = (
        state.clone()
    )

    while not rollout_state.is_terminal():

        moves = (
            rollout_state
            .get_legal_moves(
                rollout_state.current_player
            )
        )

        if not moves:

            rollout_state.pass_turn()
            continue

        move = random.choice(
            moves
        )

```

```

        rollout_state.apply_move(
            move[0],
            move[1],
            rollout_state.current_player
        )

    winner = (
        rollout_state.get_winner()
    )

    # Backpropagation
    while node is not None:

        node.visits += 1

        if winner == root_player:

            node.wins += 1

        elif winner == 0:

            node.wins += 0.5

        node = node.parent

    self.last_search_time = (
        time.time()
        - start_time
    )

    if not root.children:
        return None

    best_child = max(
        root.children,
        key=lambda child:
            child.visits
    )

    return best_child.move

```

Interfaz Gráfica (GUI)

```

In [ ]: class OthelloGUI:

    HUMAN_VS_HUMAN = 0
    HUMAN_VS_AB = 1
    HUMAN_VS_MCTS = 2
    AB_VS_MCTS = 3

    CELL_SIZE = 80
    BOARD_SIZE = 8

    GREEN = (34, 139, 34)

```

```
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
GRAY = (180, 180, 180)

def __init__(self):

    pygame.init()

    self.width = 850
    self.height = 640

    self.screen = pygame.display.set_mode(
        (self.width, self.height)
    )

    pygame.display.set_caption(
        "Othello AI"
    )

    self.font = pygame.font.SysFont(
        "Arial",
        24
    )

    self.game = OthelloEngine()

    # Cambia aquí el modo

    self.mode = self.HUMAN_VS_AB
    self.last_nodes = 0
    self.last_time = 0
    self.last_eval = 0

    self.alpha_agent = AlphaBetaAgent(
        depth=4
    )

    self.mcts_agent = MCTSAgent(
        time_limit=2.0
    )

    self.running = True

    # Tablero

    def draw_board(self):

        self.screen.fill((220, 220, 220))

        for row in range(8):
            for col in range(8):

                rect = pygame.Rect(
                    col * self.CELL_SIZE,
                    row * self.CELL_SIZE,
```

```

        self.CELL_SIZE,
        self.CELL_SIZE
    )

    pygame.draw.rect(
        self.screen,
        self.GREEN,
        rect
    )

    pygame.draw.rect(
        self.screen,
        self.BLACK,
        rect,
        1
    )

# Fichas

def draw_discs(self):

    for row in range(8):
        for col in range(8):

            value = self.game.board[row][col]

            center = (
                col * self.CELL_SIZE
                + self.CELL_SIZE // 2,

                row * self.CELL_SIZE
                + self.CELL_SIZE // 2
            )

            radius = (
                self.CELL_SIZE // 2
                - 8
            )

            if value == OthelloEngine.BLACK:

                pygame.draw.circle(
                    self.screen,
                    self.BLACK,
                    center,
                    radius
                )

            elif value == OthelloEngine.WHITE:

                pygame.draw.circle(
                    self.screen,
                    self.WHITE,
                    center,
                    radius
                )

```

```
# Movimientos válidos

def draw_legal_moves(self):

    if self.game.is_terminal():
        return

    moves = self.game.get_legal_moves(
        self.game.current_player
    )

    for row, col in moves:

        center = (
            col * self.CELL_SIZE
            + self.CELL_SIZE // 2,

            row * self.CELL_SIZE
            + self.CELL_SIZE // 2
        )

        pygame.draw.circle(
            self.screen,
            self.GRAY,
            center,
            8
        )

# Panel Lateral
def draw_info_panel(self):

    black_count, white_count = (
        self.game.count_discs()
    )

    panel_x = 660

    title = self.font.render(
        "OTHELLO",
        True,
        self.BLACK
    )

    self.screen.blit(
        title,
        (panel_x, 20)
    )

    if self.mode == self.HUMAN_VS_HUMAN:
        mode_text = "Human vs Human"

    elif self.mode == self.HUMAN_VS_AB:
        mode_text = "Human vs AlphaBeta"

    elif self.mode == self.HUMAN_VS_MCTS:
```

```
        mode_text = "Human vs MCTS"

    else:
        mode_text = "AB vs MCTS"

    mode_surface = self.font.render(
        mode_text,
        True,
        self.BLACK
    )

    self.screen.blit(
        mode_surface,
        (panel_x, 60)
    )

    nodes_surface = self.font.render(
        f"Nodes: {self.last_nodes}",
        True,
        self.BLACK
    )

    time_surface = self.font.render(
        f"Time: {self.last_time:.3f}s",
        True,
        self.BLACK
    )

    eval_surface = self.font.render(
        f"Eval: {self.last_eval:.1f}",
        True,
        self.BLACK
    )

    self.screen.blit(
        nodes_surface,
        (panel_x, 270)
    )

    self.screen.blit(
        time_surface,
        (panel_x, 310)
    )

    self.screen.blit(
        eval_surface,
        (panel_x, 350)
    )

    player_text = (
        "Black"
        if self.game.current_player
        == OthelloEngine.BLACK
        else "White"
    )
```

```
turn_surface = self.font.render(
    f"Turn: {player_text}",
    True,
    self.BLACK
)

self.screen.blit(
    turn_surface,
    (panel_x, 110)
)

black_surface = self.font.render(
    f"Black: {black_count}",
    True,
    self.BLACK
)

white_surface = self.font.render(
    f"White: {white_count}",
    True,
    self.BLACK
)

self.screen.blit(
    black_surface,
    (panel_x, 170)
)

self.screen.blit(
    white_surface,
    (panel_x, 210)
)

if self.game.is_terminal():

    winner = (
        self.game.get_winner()
    )

    if winner == OthelloEngine.BLACK:
        text = "Winner: Black"

    elif winner == OthelloEngine.WHITE:
        text = "Winner: White"

    else:
        text = "Draw"

    winner_surface = self.font.render(
        text,
        True,
        (200, 0, 0)
    )

    self.screen.blit(
        winner_surface,
```

```

        (panel_x, 300)
    )

    help1 = self.font.render(
        "1:H-H 2:H-AB",
        True,
        self.BLACK
    )

    help2 = self.font.render(
        "3:H-MCTS 4:AB-MCTS",
        True,
        self.BLACK
    )

    help3 = self.font.render(
        "R: Reset",
        True,
        self.BLACK
    )

    self.screen.blit(help1, (panel_x, 450))
    self.screen.blit(help2, (panel_x, 480))
    self.screen.blit(help3, (panel_x, 510))

# Movimiento IA

    def ai_move(self):

        if self.game.is_terminal():
            return

        current_player = self.game.current_player

        move = None

        # Human vs AlphaBeta
        if self.mode == self.HUMAN_VS_AB:

            if current_player == OthelloEngine.WHITE:

                move = self.alpha_agent.get_move(
                    self.game
                )

                self.last_nodes = (
                    self.alpha_agent.nodes_explored
                )

                self.last_time = (
                    self.alpha_agent.last_search_time
                )

                self.last_eval = (
                    OthelloHeuristic.evaluate(
                        self.game,

```



```

        current_player
    )
)

# Human vs MCTS
elif self.mode == self.HUMAN_VS_MCTS:

    if current_player == OthelloEngine.WHITE:

        move = self.mcts_agent.get_move(
            self.game
        )

        self.last_nodes = (
            self.mcts_agent.iterations_done
        )

        self.last_time = (
            self.mcts_agent.last_search_time
        )

        self.last_eval = (
            OthelloHeuristic.evaluate(
                self.game,
                current_player
            )
        )

# AlphaBeta vs MCTS
elif self.mode == self.AB_VS_MCTS:

    if current_player == OthelloEngine.BLACK:

        move = self.alpha_agent.get_move(
            self.game
        )

        self.last_nodes = (
            self.alpha_agent.nodes_explored
        )

        self.last_time = (
            self.alpha_agent.last_search_time
        )

        self.last_eval = (
            OthelloHeuristic.evaluate(
                self.game,
                current_player
            )
        )

    else:

        move = self.mcts_agent.get_move(
            self.game

```

```
        )

        self.last_nodes = (
            self.mcts_agent.iterations_done
        )

        self.last_time = (
            self.mcts_agent.last_search_time
        )

        self.last_eval = (
            OthelloHeuristic.evaluate(
                self.game,
                current_player
            )
        )

    if move:

        self.game.apply_move(
            move[0],
            move[1],
            current_player
        )

    if not self.game.has_valid_move(
        self.game.current_player
    ):

        self.game.pass_turn()

    # Click humano

    def handle_click(self, pos):

        col = pos[0] // self.CELL_SIZE
        row = pos[1] // self.CELL_SIZE

        if row >= 8 or col >= 8:
            return

        success = self.game.apply_move(
            row,
            col,
            self.game.current_player
        )

        if not success:
            return

        if not self.game.has_valid_move(
            self.game.current_player
        ):

            self.game.pass_turn()
```

```
# Resetea el juego
def reset_game(self):

    self.game = OthelloEngine()

    self.last_nodes = 0
    self.last_time = 0
    self.last_eval = 0

# Loop principal

def run(self):

    clock = pygame.time.Clock()

    while self.running:

        for event in pygame.event.get():

            if event.type == pygame.QUIT:

                self.running = False

            elif event.type == pygame.KEYDOWN:

                if event.key == pygame.K_1:

                    self.mode = self.HUMAN_VS_HUMAN
                    self.reset_game()

                elif event.key == pygame.K_2:

                    self.mode = self.HUMAN_VS_AB
                    self.reset_game()

                elif event.key == pygame.K_3:

                    self.mode = self.HUMAN_VS_MCTS
                    self.reset_game()

                elif event.key == pygame.K_4:

                    self.mode = self.AB_VS_MCTS
                    self.reset_game()

                elif event.key == pygame.K_r:

                    self.reset_game()

            elif (
                event.type
                == pygame.MOUSEBUTTONDOWN
            ):

                if (
                    self.mode
```

```

        != self.AB_VS_MCTS
        and not self.game.is_terminal()
    ):

        if (
            self.mode
            == self.HUMAN_VS_HUMAN
        ):

            self.handle_click(
                pygame.mouse.get_pos()
            )

        elif (
            self.mode
            in (
                self.HUMAN_VS_AB,
                self.HUMAN_VS_MCTS
            )
            and self.game.current_player
            == OthelloEngine.BLACK
        ):

            self.handle_click(
                pygame.mouse.get_pos()
            )

    if self.mode in (
        self.HUMAN_VS_AB,
        self.HUMAN_VS_MCTS,
        self.AB_VS_MCTS
    ):

        self.ai_move()

        self.draw_board()
        self.draw_discs()
        self.draw_legal_moves()
        self.draw_info_panel()

        pygame.display.flip()

        clock.tick(30)

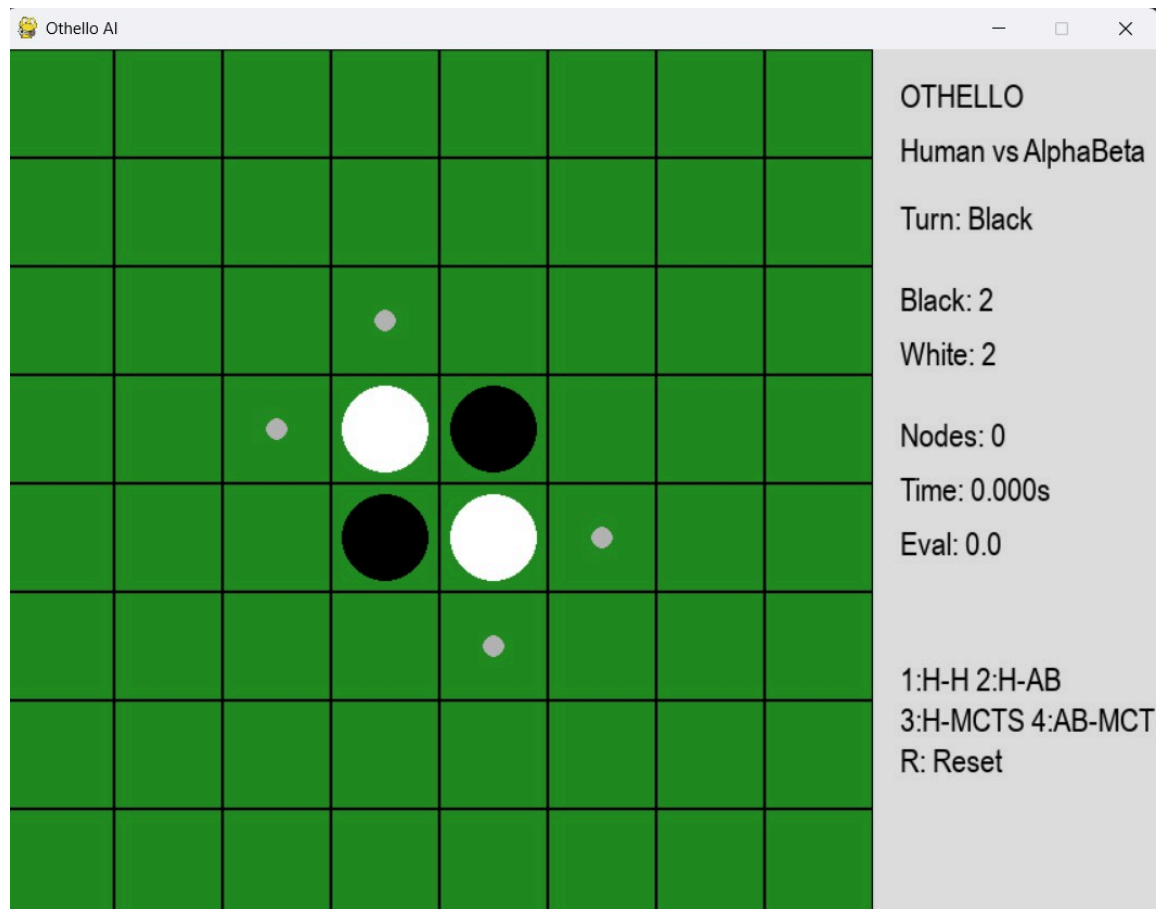
pygame.quit()

```

```

In [ ]: gui = OthelloGUI()
        gui.run()

```



Torneo Alpha-Beta vs MCTS

Con el objetivo de comparar el desempeño de ambos algoritmos, se implementa un torneo automático entre los agentes Alpha-Beta y Monte Carlo Tree Search (MCTS). Las partidas se ejecutan sin intervención humana y se registran métricas como victorias, empates y tiempo promedio de decisión. Estos resultados servirán posteriormente para el análisis de rendimiento requerido por el proyecto.

```
In [ ]: def run_tournament(
    games=20,
    ab_depth=4,
    mcts_iterations=100
):

    results = []

    alpha_wins = 0
    mcts_wins = 0
    draws = 0

    total_ab_time = 0
    total_mcts_time = 0

    for game_number in range(games):
```

```
game = OthelloEngine()

alpha_agent = AlphaBetaAgent(
    depth=ab_depth
)

mcts_agent = MCTSAgent(
    time_limit=2.0
)

while not game.is_terminal():

    current_player = (
        game.current_player
    )

    legal_moves = (
        game.get_legal_moves(
            current_player
        )
    )

    if not legal_moves:

        game.pass_turn()
        continue

    # AlphaBeta = Negro

    if current_player == OthelloEngine.BLACK:

        move = (
            alpha_agent.get_move(
                game
            )
        )

        total_ab_time += (
            alpha_agent
            .last_search_time
        )

    # MCTS = Blanco

    else:

        move = (
            mcts_agent.get_move(
                game
            )
        )

        total_mcts_time += (
            mcts_agent
            .last_search_time
```

```

    )

    if move:

        game.apply_move(
            move[0],
            move[1],
            current_player
        )

    winner = game.get_winner()

    if winner == OthelloEngine.BLACK:

        alpha_wins += 1

        result = "AlphaBeta"

    elif winner == OthelloEngine.WHITE:

        mcts_wins += 1

        result = "MCTS"

    else:

        draws += 1

        result = "Draw"

    black_count, white_count = (
        game.count_discs()
    )

    results.append({
        "Game": game_number + 1,
        "Winner": result,
        "Black": black_count,
        "White": white_count
    })

    print(
        f"Game {game_number+1}: {result}"
    )

summary = {
    "AlphaBeta Wins": alpha_wins,
    "MCTS Wins": mcts_wins,
    "Draws": draws,
    "Average AlphaBeta Time":
        total_ab_time /
        max(alpha_wins + mcts_wins + draws, 1),

    "Average MCTS Time":
        total_mcts_time /
        max(alpha_wins + mcts_wins + draws, 1)
}

```

```
}  
  
return (  
    pd.DataFrame(results),  
    summary  
)
```

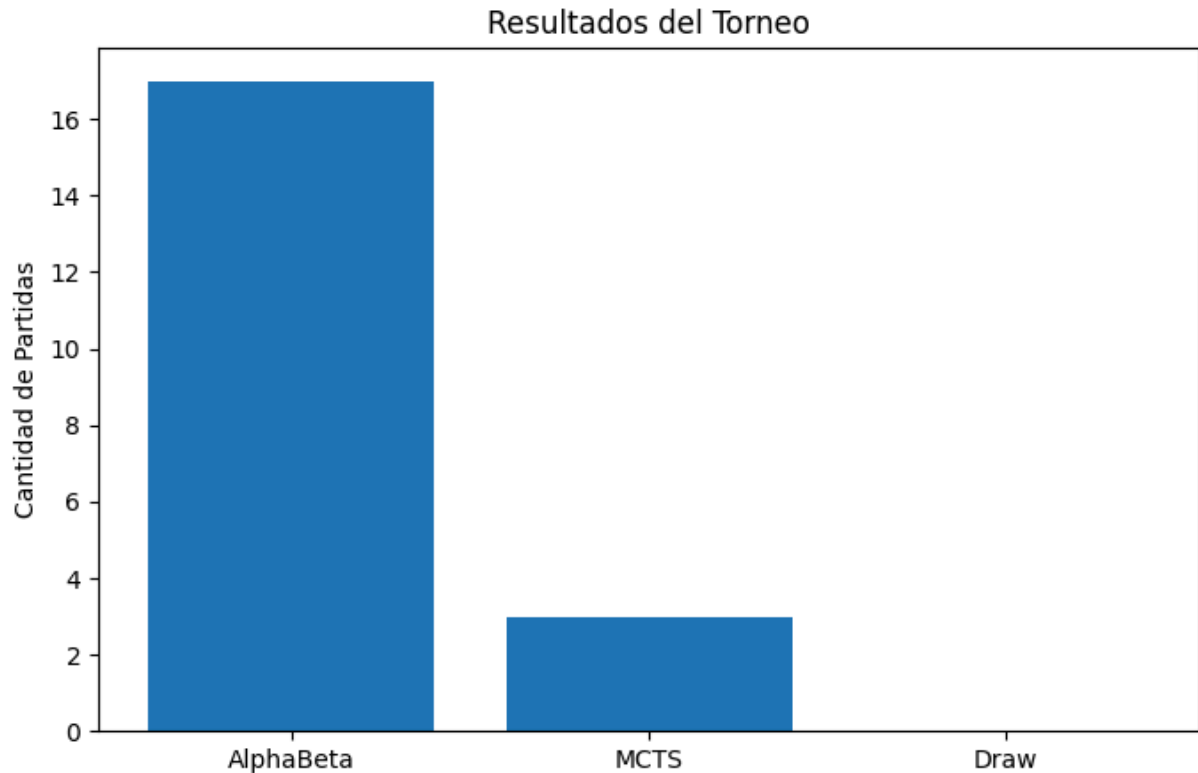
```
In [ ]: results_df, summary = run_tournament(  
        games=20,  
        ab_depth=4,  
        mcts_iterations=100  
    )  
  
print(summary)  
  
results_df
```

Análisis y Gráficas

Análisis de Rendimiento

Con el objetivo de evaluar el comportamiento de los agentes implementados, se realizaron múltiples partidas automáticas entre Alpha-Beta y Monte Carlo Tree Search (MCTS). Además de registrar victorias y derrotas, se analizaron métricas relacionadas con el tiempo de búsqueda y la complejidad de exploración del árbol de juego. Estos resultados permiten comparar la eficiencia y efectividad de ambos enfoques para la toma de decisiones en Othello.

```
In [ ]: import matplotlib.pyplot as plt  
  
alpha_wins = summary["AlphaBeta Wins"]  
mcts_wins = summary["MCTS Wins"]  
draws = summary["Draws"]  
  
plt.figure(figsize=(8,5))  
  
plt.bar(  
    ["AlphaBeta", "MCTS", "Draw"],  
    [alpha_wins, mcts_wins, draws]  
)  
  
plt.title(  
    "Resultados del Torneo"  
)  
  
plt.ylabel(  
    "Cantidad de Partidas"  
)  
  
plt.show()
```

Resultados del Torneo

La siguiente gráfica muestra la distribución de victorias obtenidas por cada agente durante las partidas automáticas realizadas. Esta comparación permite identificar cuál de los algoritmos logró un mejor desempeño general bajo las mismas condiciones de juego.

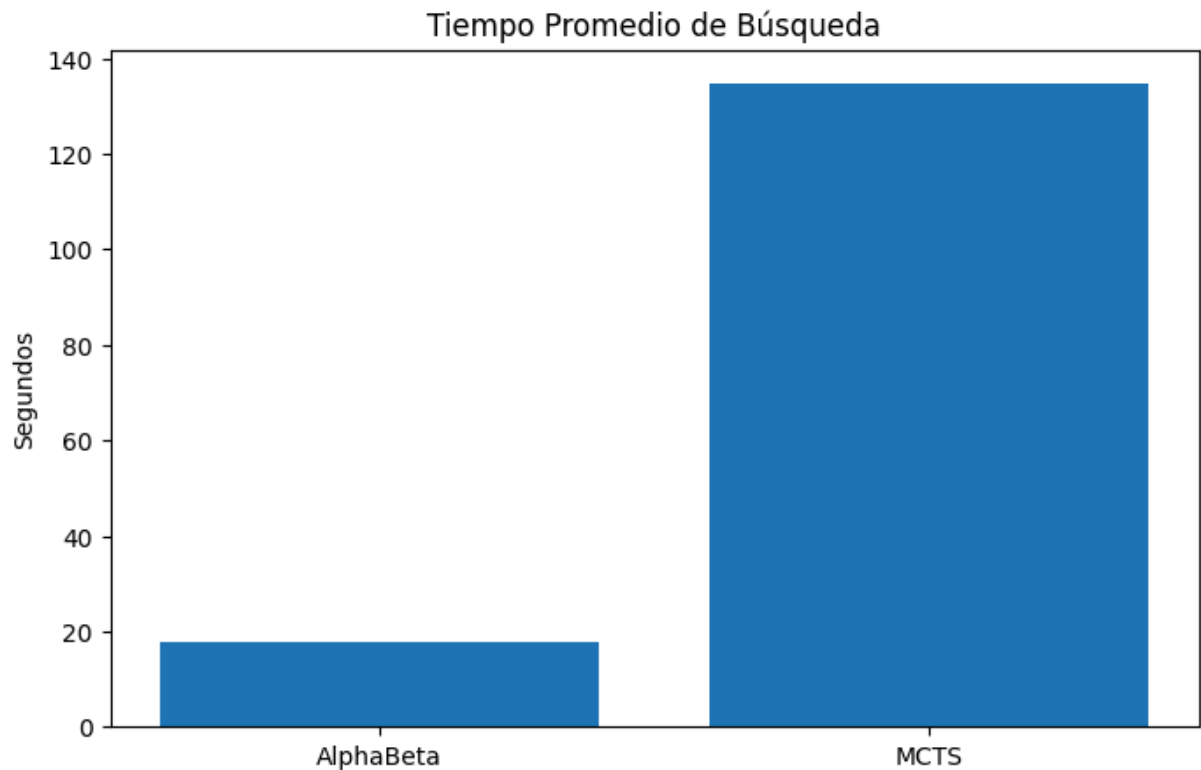
```
In [ ]: # Grafica de tiempo promedio
plt.figure(figsize=(8,5))

plt.bar(
    ["AlphaBeta", "MCTS"],
    [
        summary["Average AlphaBeta Time"],
        summary["Average MCTS Time"]
    ]
)

plt.title(
    "Tiempo Promedio de Búsqueda"
)

plt.ylabel(
    "Segundos"
)

plt.show()
```



```
In [ ]: # Grafica de profundidad vs nodos
depths = [1, 2, 3, 4]

nodes = []

for depth in depths:

    game = OthelloEngine()

    agent = AlphaBetaAgent(
        depth=depth
    )

    agent.get_move(game)

    nodes.append(
        agent.nodes_explored
    )

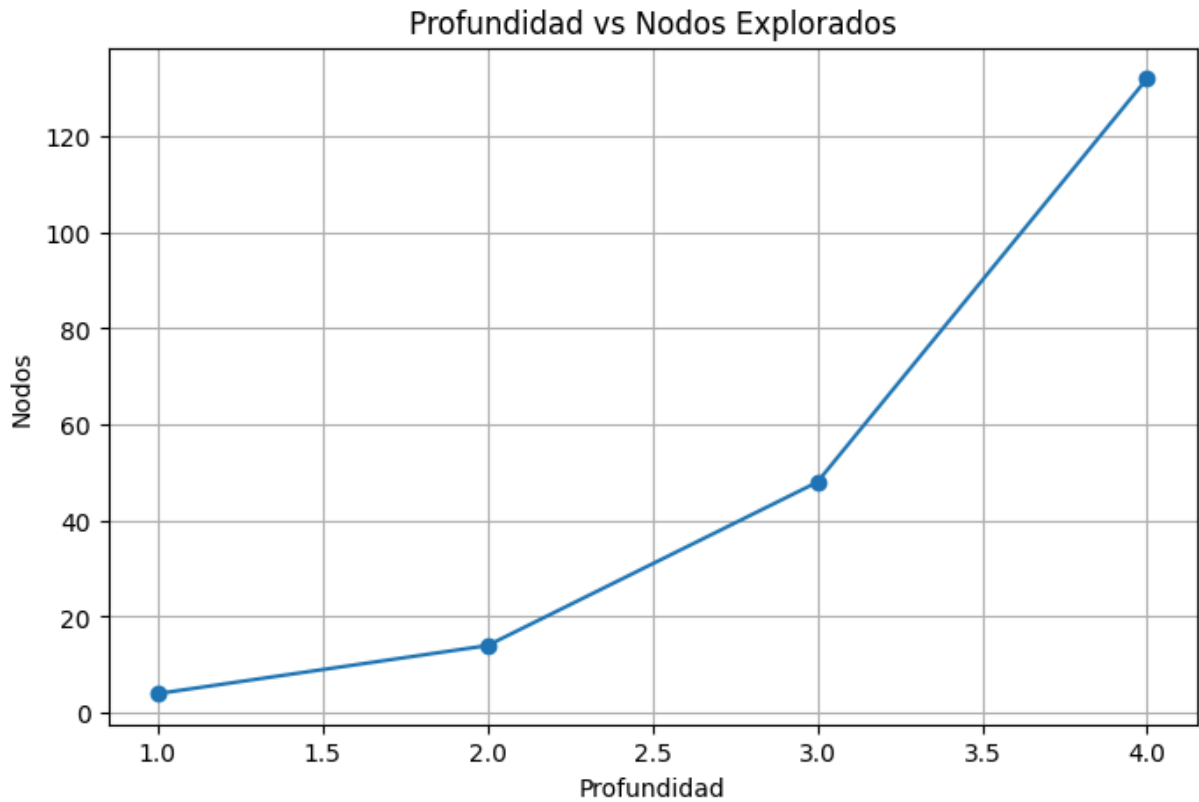
plt.figure(figsize=(8,5))

plt.plot(
    depths,
    nodes,
    marker="o"
)

plt.title(
    "Profundidad vs Nodos Explorados"
)

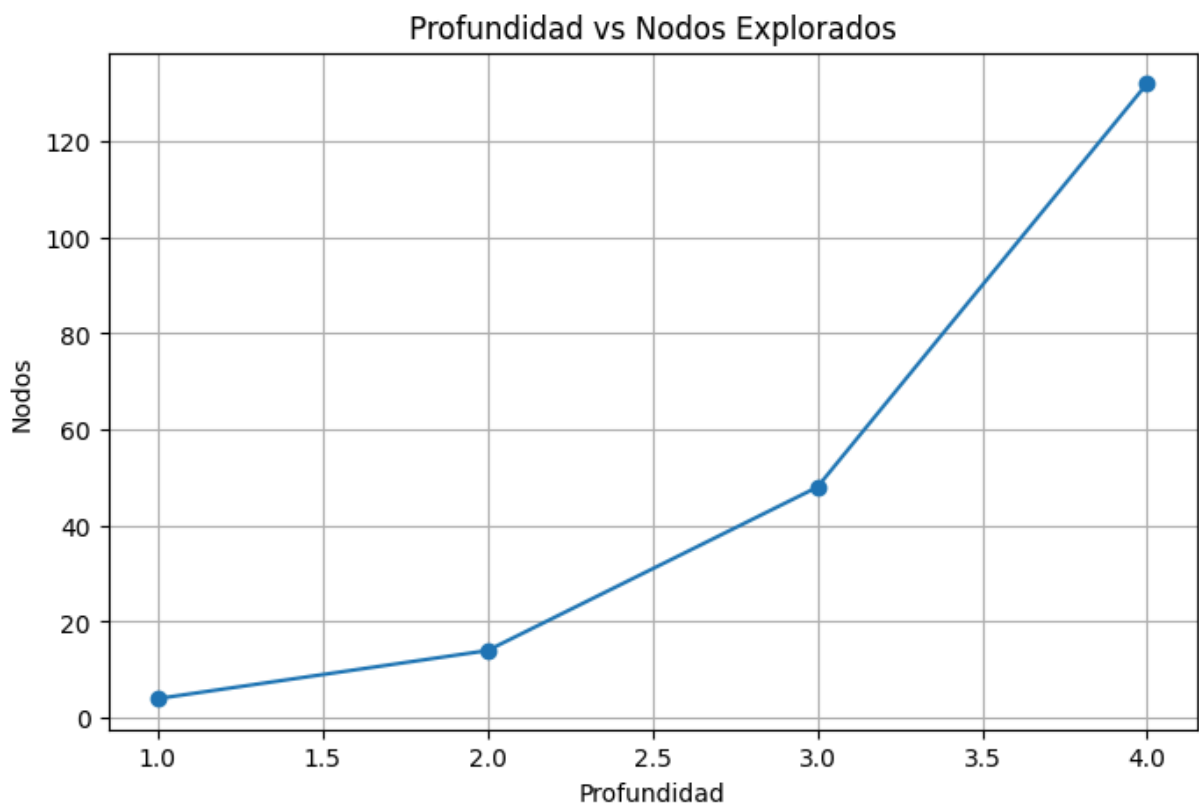
plt.xlabel(
```

```
    "Profundidad"  
)  
plt.ylabel(  
    "Nodos"  
)  
plt.grid(True)  
plt.show()
```



```
In [ ]: # Grafica de profundidad vs nodos  
depths = [1, 2, 3, 4]  
  
nodes = []  
  
for depth in depths:  
    game = OthelloEngine()  
    agent = AlphaBetaAgent(  
        depth=depth  
    )  
    agent.get_move(game)  
    nodes.append(  
        agent.nodes_explored  
    )  
  
plt.figure(figsize=(8,5))
```

```
plt.plot(  
    depths,  
    nodes,  
    marker="o"  
)  
  
plt.title(  
    "Profundidad vs Nodos Explorados"  
)  
  
plt.xlabel(  
    "Profundidad"  
)  
  
plt.ylabel(  
    "Nodos"  
)  
  
plt.grid(True)  
  
plt.show()
```



Comparacion Alphabeta vs Minimax

```
In [ ]: depths = [1, 2, 3, 4]  
  
minimax_nodes = []  
alphabeta_nodes = []
```

```
for depth in depths:

    game = OthelloEngine()

    minimax = MinimaxAgent(
        depth=depth
    )

    alphabeta = AlphaBetaAgent(
        depth=depth
    )

    minimax.get_move(game)

    alphabeta.get_move(game)

    minimax_nodes.append(
        minimax.nodes_explored
    )

    alphabeta_nodes.append(
        alphabeta.nodes_explored
    )
```

```
In [ ]: # Grafica
plt.figure(figsize=(8,5))

plt.plot(
    depths,
    minimax_nodes,
    marker="o",
    label="Minimax"
)

plt.plot(
    depths,
    alphabeta_nodes,
    marker="o",
    label="Alpha-Beta"
)

plt.title(
    "Minimax vs Alpha-Beta"
)

plt.xlabel(
    "Profundidad"
)

plt.ylabel(
    "Nodos Explorados"
)

plt.legend()

plt.grid(True)
```

```
plt.show()
```

